

boston_analysis

May 19, 2026

1 —

Boston Housing UCI / OpenML 506 13
MEDV
BaselineLR RandomForest XGBoost LightGBM
MSE MAE R^2

1.1 1.

```
[91]: import os
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.datasets import fetch_openml
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
plt.rcParams.update({'figure.dpi': 120, 'savefig.dpi': 150, 'savefig.bbox': 'tight',
                    'font.sans-serif': ['DejaVu Sans']})

# notebook Jupyter
_HERE = Path(os.getcwd()).resolve()
if not (_HERE / 'config.py').exists():
    _alt = _HERE / 'boston_housing'
    if (_alt / 'config.py').exists():
        os.chdir(str(_alt))
    _HERE = _alt
```

```

RANDOM_STATE = 42
TEST_SIZE = 0.2

OUTPUT_DIR = _HERE / 'output'
(OUTPUT_DIR / 'plots').mkdir(parents=True, exist_ok=True)
print(f'    {_HERE}')
print('    ')

/home/zxstring/project/aiInd/boston_housing

```

1.2 2.

OpenML	506	1970	13	MEDV
--------	-----	------	----	------

```

[92]: bunch = fetch_openml(name='boston', version=1, as_frame=True, parser='auto')
df = bunch.frame
df.columns = [c.upper() for c in df.columns]

#
for c in df.columns:
    if df[c].dtype.name in ('category', 'object'):
        df[c] = pd.to_numeric(df[c], errors='coerce')

print(f'    {df.shape[0]} × {df.shape[1]} ')
print(f'    {list(df.columns[:-1])}')
print(f'    MEDV    / $1000    [{df.MEDV.min():.0f}, {df.MEDV.max():.0f}]')

506 × 14
['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
'PTRATIO', 'B', 'LSTAT']
MEDV    / $1000    [5, 50]

```

1.3 3.

```

[93]: #
missing = df.isnull().sum()
if missing.sum() == 0:
    print('    ')
else:
    print(missing[missing > 0])

```

```
[94]: #
df.describe().T.round(2)
```

```
[94]:
```

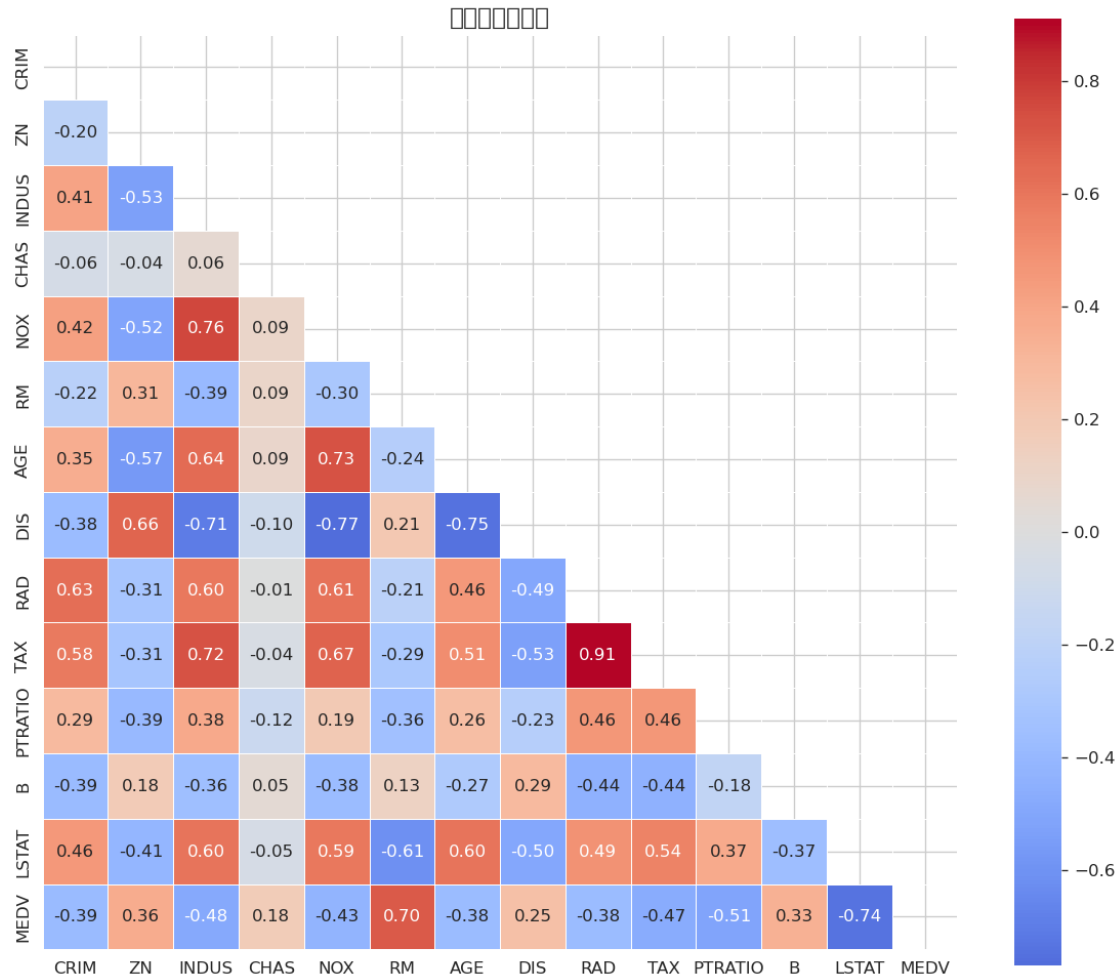
	count	mean	std	min	25%	50%	75%	max
CRIM	506.0	3.61	8.60	0.01	0.08	0.26	3.68	88.98
ZN	506.0	11.36	23.32	0.00	0.00	0.00	12.50	100.00
INDUS	506.0	11.14	6.86	0.46	5.19	9.69	18.10	27.74
CHAS	506.0	0.07	0.25	0.00	0.00	0.00	0.00	1.00
NOX	506.0	0.55	0.12	0.38	0.45	0.54	0.62	0.87
RM	506.0	6.28	0.70	3.56	5.89	6.21	6.62	8.78
AGE	506.0	68.57	28.15	2.90	45.02	77.50	94.07	100.00
DIS	506.0	3.80	2.11	1.13	2.10	3.21	5.19	12.13
RAD	506.0	9.55	8.71	1.00	4.00	5.00	24.00	24.00
TAX	506.0	408.24	168.54	187.00	279.00	330.00	666.00	711.00
PTRATIO	506.0	18.46	2.16	12.60	17.40	19.05	20.20	22.00
B	506.0	356.67	91.29	0.32	375.38	391.44	396.22	396.90
LSTAT	506.0	12.65	7.14	1.73	6.95	11.36	16.96	37.97
MEDV	506.0	22.53	9.20	5.00	17.02	21.20	25.00	50.00

```

      - CRIM      3.61      88.98——      - ZN      0      -
B      - MEDV    22.5    21.2      50

```

```
[95]: #
fig, ax = plt.subplots(figsize=(12, 10))
corr = df.corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, square=True, linewidths=0.5, ax=ax)
ax.set_title(' ', fontsize=14)
fig.savefig(OUTPUT_DIR / 'plots' / 'correlation_heatmap.png')
plt.show()
```



- RAD TAX 0.91 ——— - DIS NOX / INDUS
 / AGE ——— - MEDV LSTAT -0.74 RM 0.70 ———

```
[96]: # MEDV +
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

axes[0].hist(df['MEDV'], bins=30, edgecolor='k', alpha=0.7)
axes[0].axvline(df['MEDV'].mean(), color='#e74c3c', ls='--', lw=1.5,
                label=f'={df.MEDV.mean():.1f}')
axes[0].axvline(df['MEDV'].median(), color='#2ecc71', ls='--', lw=1.5,
                label=f'={df.MEDV.median():.1f}')
axes[0].set_xlabel('MEDV ')
axes[0].set_title('MEDV ')
axes[0].legend()

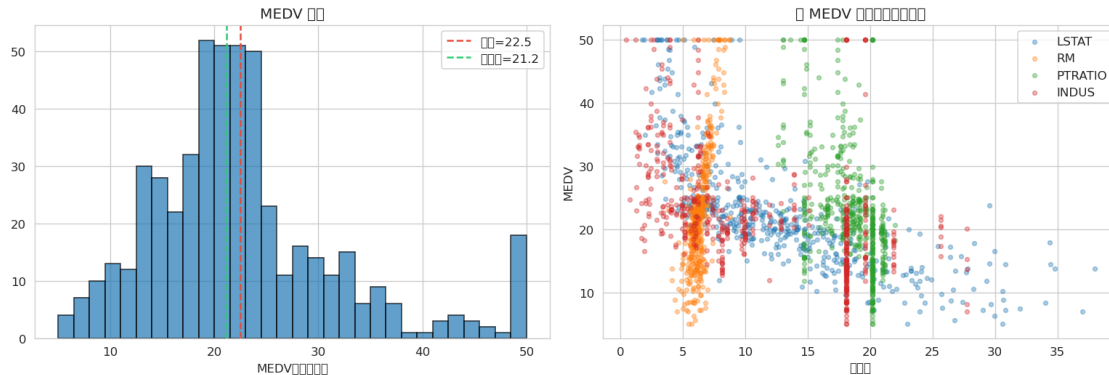
top4 = corr['MEDV'].abs().sort_values(ascending=False).index[1:5]
```

```

for feat in top4:
    axes[1].scatter(df[feat], df['MEDV'], alpha=0.35, s=12, label=feat)
axes[1].set_xlabel(' ')
axes[1].set_ylabel('MEDV')
axes[1].set_title(' MEDV ')
axes[1].legend()

plt.tight_layout()
plt.show()

```



MEDV 50 " " capped at 50 50k 50 — >50

5-10

1.4 4.

1.4.1 4.1 — Winsorize

IQR 1.5 — " " 506

```

[97]: def winsorize(series, limit=1.5):
    q1, q3 = series.quantile(0.25), series.quantile(0.75)
    return series.clip(q1 - limit * (q3 - q1), q3 + limit * (q3 - q1))

cols_to_winsor = ['CRIM', 'RM', 'B', 'ZN', 'DIS']
for col in cols_to_winsor:
    if col in df.columns:
        q1, q3 = df[col].quantile(0.25), df[col].quantile(0.75)
        iqr = q3 - q1
        before = ((df[col] < q1 - 1.5 * iqr) | (df[col] > q3 + 1.5 * iqr)).sum()
        df[col] = winsorize(df[col])
        print(f'{col} {before} → ')

```

CRIM 66 →

RM 30 →

B 77 →
 ZN 68 →
 DIS 5 →

1.4.2 4.2

- RM^2 ——— - $RM \times LSTAT$ ———
 - $DIS \times NOX$ ——— vs

```
[98]: df['RM2'] = df['RM'] ** 2
df['RM_LSTAT'] = df['RM'] * df['LSTAT']
df['DIS_NOX'] = df['DIS'] * df['NOX']

y = df['MEDV'].values
X = df.drop(columns=['MEDV'])

print(f' {X.shape[1]} 13 + 3 ')
```

16 13 + 3

1.4.3 4.3

80/20 42

```
[99]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE)

print(f' {X_train.shape[0]} | {X_test.shape[0]} | {X_train.
↪shape[1]} ')
```

404 | 102 | 16

1.5 5.

1.5.1 5.1

4

	BaselineLR	
Bagging	RandomForest	+ Bootstrap +
Boosting	XGBoost	+
Boosting	LightGBM	XGBoost leaf-wise +
		GOSS

1.5.2 5.2

RandomizedSearchCV - BaselineLR fit_intercept 1 - RandomForest 30 -
 XGBoost / LightGBM 200 ShuffleSplit
 RandomForest 120s Boosting 900s fallback

1.5.3 5.3

4 10 seed 43-52 R^2 MSE

1.6 6. seed=45

output_s45/results.csv train.py + run.sh seed=45 10 XGBoost

```
[100]: results_df = pd.read_csv(_HERE / 'output_s45' / 'results.csv')
results_df = results_df.sort_values('mse')

#
display_cols = ['model', 'mse', 'mae', 'r2', 'cv_mse', 'time_s']
display = results_df[display_cols].copy()
display.columns = [' ', 'MSE', 'MAE', 'R2', 'CV MSE', ' (s)']
display.style.format({
    'MSE': '{:.2f}', 'MAE': '{:.2f}', 'R2': '{:.4f}', 'CV MSE': '{:.2f}',
    ' (s)': '{:.0f}'
}).set_caption(' seed=45 MSE ')
```

[100]: <pandas.io.formats.style.Styler at 0x76299858ac10>

1.6.1 6.1

XGBoost $R^2=0.9325$ MSE=7.29 LightGBM 1.2 seed=45 XGBoost level-wise +

LightGBM $R^2=0.9201$ XGBoost 1/4 160s vs 719s LightGBM

RandomForest $R^2=0.9019$ 0.9 R^2 Bagging " " Boosting

BaselineLR $R^2=0.8718$ 0.87 Ridge 4

RandomForest timed_out RandomizedSearchCV 120s 0.9 R^2

```
[101]: #
fig, axes = plt.subplots(1, 3, figsize=(14, 4))

metrics = [
    ('mse', 'MSE '),
    ('mae', 'MAE '),
    ('r2', 'R2 '),
]

for ax, (metric, title) in zip(axes, metrics):
    vals = results_df[metric].values
    names = results_df['model'].values

    if metric == 'r2':
```

```

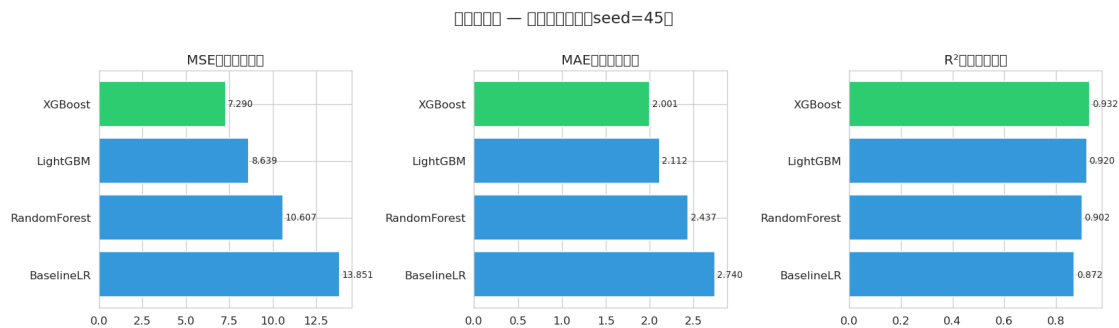
        best_idx = np.argmax(vals)
    else:
        best_idx = np.argmin(vals)

    colors = ['#2ecc71' if i == best_idx else '#3498db' for i in
    ↪range(len(vals))]
    ax.barh(names, vals, color=colors)
    ax.set_title(title, fontsize=12)
    ax.invert_yaxis()

    for i, v in enumerate(vals):
        offset = max(vals) * 0.01
        ax.text(v + offset, i, f'{v:.3f}', va='center', fontsize=8)

fig.suptitle(' - seed=45 ', fontsize=14, y=1.01)
fig.tight_layout()
fig.savefig(OUTPUT_DIR / 'plots' / 'model_comparison.png')
plt.show()

```



1.7 7.

vs

6

```

[102]: quick_models = {
        'BaselineLR': LinearRegression(),
        'RandomForest': RandomForestRegressor(n_estimators=100, max_depth=8,
                                              random_state=42, n_jobs=-1),
        'XGBoost': XGBRegressor(n_estimators=100, max_depth=4, learning_rate=0.1,
                                random_state=42, tree_method='hist', verbosity=0),
        'LightGBM': LGBMRegressor(n_estimators=100, max_depth=4, learning_rate=0.1,
                                   random_state=42, verbose=-1),
    }

```

```

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()

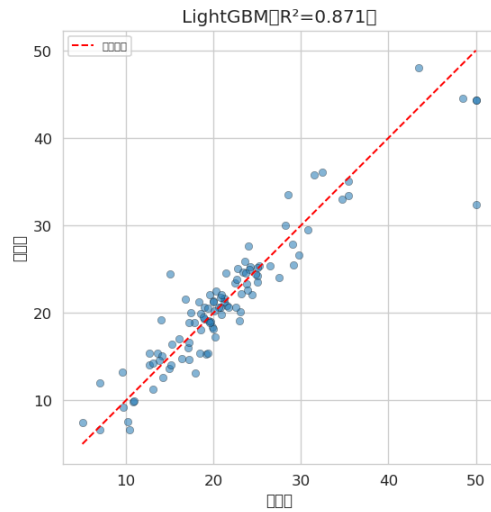
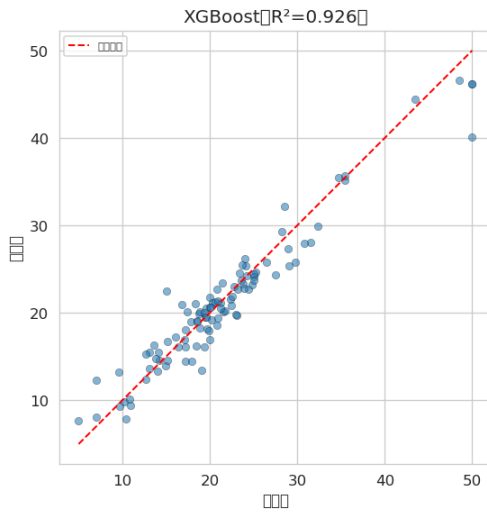
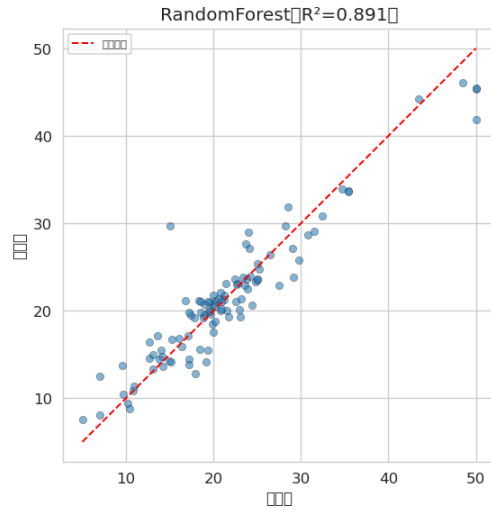
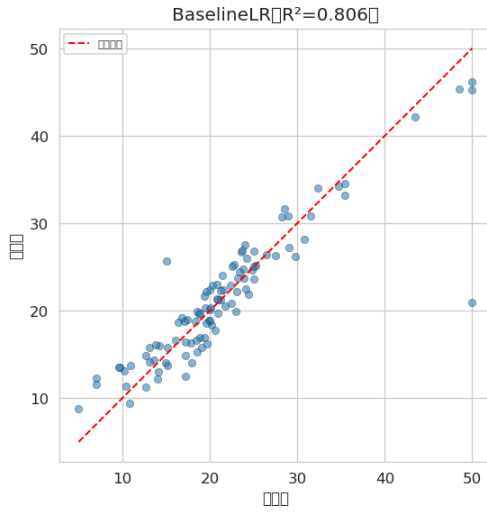
for i, (name, est) in enumerate(quick_models.items()):
    ax = axes[i]
    est.fit(X_train, y_train)
    yp = est.predict(X_test)
    r2 = r2_score(y_test, yp)

    ax.scatter(y_test, yp, alpha=0.55, s=25, edgecolors='k', linewidth=0.3)
    ax.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
            'r--', lw=1.2, label=' ')
    ax.set_xlabel(' ')
    ax.set_ylabel(' ')
    ax.set_title(f'{name} R²={r2:.3f} ')
    ax.legend(fontsize=7)
    ax.set_aspect('equal')

fig.suptitle(' vs ', fontsize=13, y=1.01)
fig.tight_layout()
fig.savefig(OUTPUT_DIR / 'plots' / 'predictions_vs_actual.png')
plt.show()

```

□□□ VS □□□□□□□□□□□□□□□□



” ” - BaselineLR 40-50 — RandomForest
50 — MEDV - XGBoost LightGBM 20-35
Boosting

1.8 8.

	XGBoost	BaselineLR / RandomForest / XGBoost / LightGBM	10
seed 43-52			

```
[103]: # output_s*/ summary_all.csv
records = []
for seed in range(43, 53):
    f = _HERE / f'output_s{seed}' / 'results.csv'
    if f.exists():
```

```

df_s = pd.read_csv(f)
df_s = df_s[['model', 'mse', 'mae', 'r2', 'cv_mse', 'time_s']].copy()
df_s.insert(0, 'seed', seed)
records.append(df_s)
multi = pd.concat(records, ignore_index=True)
print(f' {multi.seed.nunique()} {multi.model.nunique()} {len(multi)} ')

#
agg = multi.groupby('model').agg(
    r2_mean=('r2', 'mean'), r2_std=('r2', 'std'),
    mse_mean=('mse', 'mean'), mse_std=('mse', 'std'),
    mae_mean=('mae', 'mean'), mae_std=('mae', 'std'),
).round(4)
agg

```

```
10    4    40
```

```
[103]:
```

	r2_mean	r2_std	mse_mean	mse_std	mae_mean	mae_std
model						
BaselineLR	0.7996	0.0388	17.8392	4.1165	2.7810	0.2260
LightGBM	0.8858	0.0271	10.1865	2.9644	2.1690	0.1870
RandomForest	0.8845	0.0224	10.3101	2.5763	2.2339	0.2161
XGBoost	0.8959	0.0214	9.2390	2.1865	2.1088	0.1768

```
[104]: # - 10 R² MSE
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))

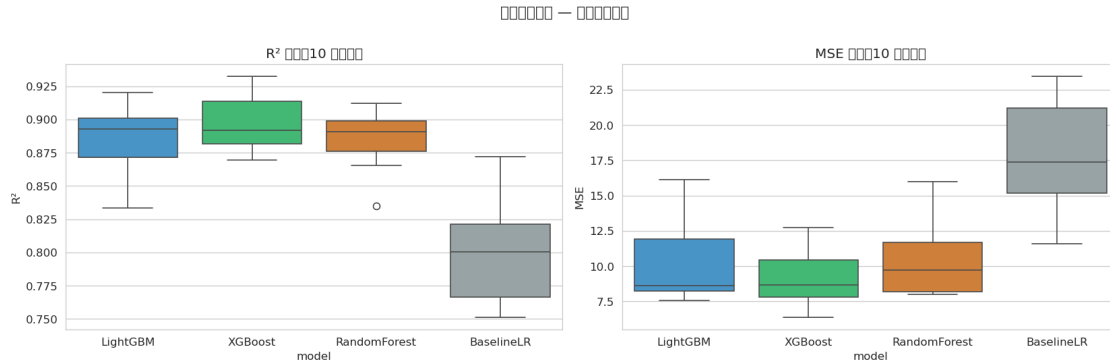
order_r2 = multi.groupby('model')['r2'].median().sort_values(ascending=False).
    .index.tolist()
palette_list = {m: c for m, c in [('XGBoost', '#2ecc71'), ('LightGBM', '#3498db'),
    ('RandomForest', '#e67e22'), ('BaselineLR', '#95a5a6')]}

sns.boxplot(data=multi, x='model', y='r2', order=order_r2,
            palette=[palette_list[m] for m in order_r2], ax=axes[0])
axes[0].set_title('R² 10 ')
axes[0].set_ylabel('R²')

sns.boxplot(data=multi, x='model', y='mse', order=order_r2,
            palette=[palette_list[m] for m in order_r2], ax=axes[1])
axes[1].set_title('MSE 10 ')
axes[1].set_ylabel('MSE')

fig.suptitle(' - ', fontsize=13, y=1.01)
fig.tight_layout()
plt.show()

```



10

- **XGBoost** R^2 **0.896** ± 0.021 9/10 seed=43 LightGBM 0.9181
 $R^2=0.9325$ seed=45 0.8693 seed=49
- **LightGBM** **0.886** **RandomForest** **0.885** LightGBM ± 0.027 vs
 ± 0.022 — leaf-wise LightGBM 0.9201 RF
- **BaselineLR** R^2 **0.80** ± 0.039 —
- **XGBoost** Boosting +

1.9 9.

RF impurity XGBoost gain/weight/cover

RandomForest XGBoost

```
[105]: feature_names = X.columns.tolist()

fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))

models_for_fi = [
    ('RandomForest', RandomForestRegressor(n_estimators=200, max_depth=10,
                                           random_state=42, n_jobs=-1)),
    ('XGBoost', XGBRegressor(n_estimators=200, max_depth=5, learning_rate=0.05,
                             random_state=42, tree_method='hist', verbosity=0)),
]

for ax, (name, est) in zip(axes, models_for_fi):
    est.fit(X_train, y_train)
    importances = est.feature_importances_
    idx = np.argsort(importances)[::-1][:15]

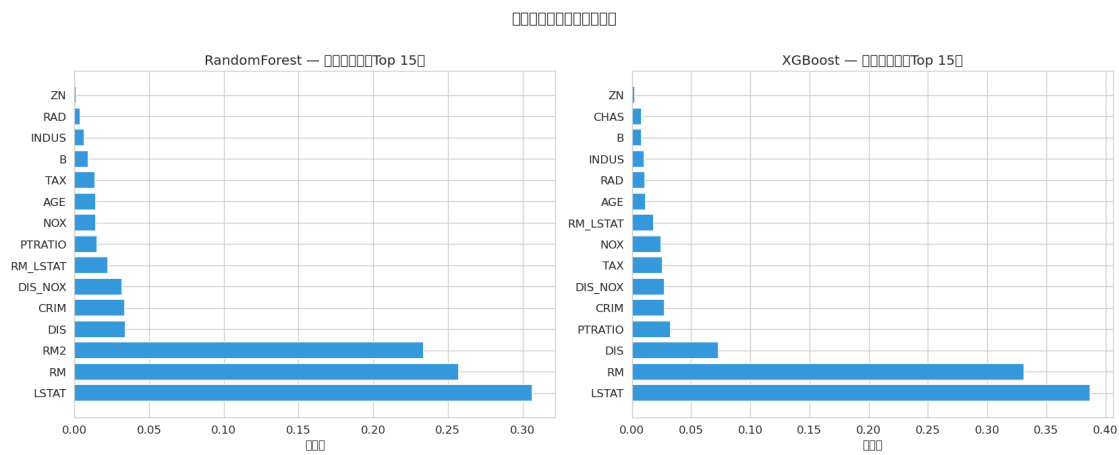
    ax.barh(range(len(idx)), importances[idx][::-1], color='#3498db')
    ax.set_yticks(range(len(idx)))
    ax.set_yticklabels([feature_names[i] for i in idx][::-1])
    ax.set_xlabel('')
```

```

ax.set_title(f'{name} - Top 15 ')
ax.invert_yaxis()

fig.suptitle(' ', fontsize=13, y=1.01)
fig.tight_layout()
plt.show()

```



Top LSTAT RM
 - RM2 Top 5 - DIS_NOX ×
 ” - XGBoost Top 5 LSTAT RM RM_LSTAT DIS NOX
 CHAS

1.10 10.

1.10.1 10.1

1. **XGBoost** — $R^2=0.9325$ seed=45 10 $R^2=0.896$ ± 0.021 Boosting
2. — RM^2 RM_LSTAT
3. **Boosting Bagging** — LightGBM 0.886 RandomForest 0.885 10 Boosting
4. — BaselineLR $R^2=0.80$ 10

1.10.2 10.2

- 506 Boosting
- **MEDV** 50k
- 13 R^2 0.93 ” ”
- **RandomForest** RF 120s

1.10.3 10.3

- log(MEDV) exp — - XGBoost early stopping 10%
n_estimators - Bayesian optimization RandomizedSearchCV

train.py *config.py* *run.sh* *run_multi.sh*